Assignment 3: Modern Neural Network Architectures and Object Detection

1 Part A: Image Classification with Residual + Inception + Fire

We implemented three reusable PyTorch modules — **ResidualBlock** (Conv-BN-ReLU → Conv-BN + skip), **InceptionBlock** (parallel 1×1, 3×3, 5×5, and pooling branches concatenated), and **FireModule** (squeeze 1×1 → expand 1×1 and 3×3 concatenated) — and used them to build five models trained on CIFAR-10 for 20 epochs with Adam (lr = 10^{-3}) and cosine annealing.

Model	Parameters	Best Test Acc
Plain CNN (baseline)	288,746	0.8644
ResNet-like	575,274	**0.8867**
Inception-like	310,250	0.8554
SqueezeNet-like	40,394	0.7405
SuperNet (hybrid)	227,594	0.8670

Table 1: Part A: CIFAR-10 test accuracy after 20 epochs.

The **ResNet-like** model achieved the highest accuracy (88.7%) but used the most parameters (575k). Skip connections allowed it to train deeper without degradation, consistent with our findings in Assignment 1. The **Inception-like** model (85.5%) slightly underperformed the plain CNN despite having multi-scale branches; with only 20 epochs the parallel pathways may not have fully converged. **SqueezeNet-like** had by far the fewest parameters (40k) but paid a significant accuracy penalty (74.1%), showing that aggressive compression via squeeze layers limits representational capacity on a dataset as varied as CIFAR-10.

The **SuperNet** is the most interesting result: it combines all three modules — Inception at the front for multi-scale feature extraction, Residual blocks in the middle for deep refinement with stable gradients, and a Fire module at the end for parameter-efficient final features. With only 228k parameters (60% fewer than ResNet-like), it achieved 86.7%, matching the plain CNN and approaching ResNet-like performance. This demonstrates that combining architectural ideas can yield a good accuracy-to-parameter ratio.

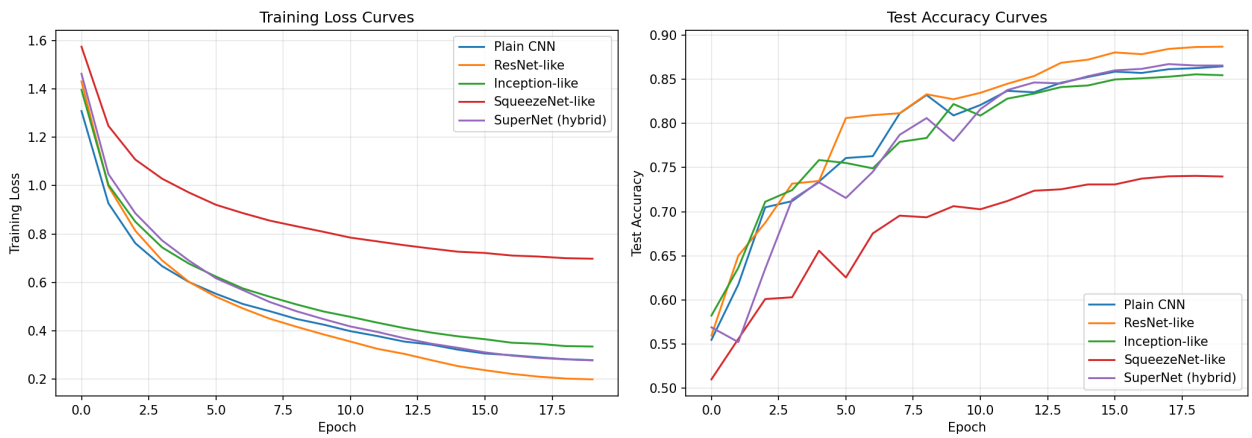

Figure 1: Training loss and test accuracy over 20 epochs for all five models.

The training curves confirm that ResNet-like converges fastest and to the lowest loss, while SqueezeNet-like converges slowest. SuperNet's loss curve closely tracks the plain CNN but achieves slightly better generalization, validating the hybrid design.

2 Part B: Object Detection — YOLO vs RT-DETR

We compared **YOLOv8n** (3.2M params, CNN-based single-stage detector) and **RT-DETR-L** (32.1M params, transformer-based detector) on the COCO128 subset (128 images, 929 object instances) using pretrained weights without any training.

Model	Parameters	GFLOPs	mAP@0.5	mAP@0.5:95
YOLOv8n	3.2M	8.7	0.607	0.448
RT-DETR-L	32.1M	103.8	**0.844**	**0.651**

Table 2: Part B: Object detection mAP on COCO128.

RT-DETR-L dramatically outperformed YOLOv8n across both metrics (mAP@0.5: 0.844 vs 0.607). This is expected given the 10× parameter and compute gap, but it also reflects a fundamental architectural difference: RT-DETR uses self-attention to reason about global image context, while YOLOv8n relies on local convolutional features.

Failure analysis. We ran both models on all 128 images and compared per-image detection counts. YOLO missed more objects than RT-DETR in 78 out of 128 images, while RT-DETR missed more in only 1 image. Analysing the top-10 YOLO failure cases revealed two clear trends: all 10 were dominated by *small objects* (relative area <1% of the image) and all 10 were *crowded scenes* (>8 objects). RT-DETR's attention mechanism allows it to aggregate global context, making it better at detecting small and occluded objects in dense scenes. YOLO's CNN backbone processes local neighbourhoods, so it tends to merge or miss small nearby objects.

The trade-off is speed: YOLOv8n ran at 5.8ms per image versus RT-DETR-L at 16.3ms — nearly 3× slower. For real-time applications where speed matters more than detecting every small object, YOLO remains the practical choice. For applications requiring high recall on small or occluded objects (e.g. medical imaging, surveillance), transformer-based detectors like RT-DETR are superior.